CLAPCAP: LIGHTWEIGHT LANGUAGE MODELS FOR MUSIC CAPTIONING VIA A CLAP-TO-LM PREFIX PIPELINE

Ali Ozkaya
Izmir Institute of Technology
aliozkaya00@gmail.com

ABSTRACT

We study whether a *lightweight* language model (LM) can describe music when fed nothing but a short, learned prefix projected from a frozen audio embedding. Our pipeline, CLAPCAP, is the audio analogue of ClipCap [1]: a frozen CLAP audio encoder produces a single clip embedding, a small projection network maps it to a fixed-length prefix of pseudo-tokens, and a pretrained LM autoregressively generates the caption. On MusicCaps [2] we run a controlled comparison of two LM families — GPT-2 (decoder-only, 124M) and T5-base (encoder–decoder, 220M) — holding the encoder, projection, data split and evaluation fixed so that the LM is the only variable. We report a ten-metric suite with FENSE [5] as the primary score, and we measure a per-model *noise floor* from three seeds so that every ablation delta can be judged against run-to-run variance. We find that (i) the two LMs are statistically indistinguishable at baseline once the noise floor is accounted for; (ii) second-stage fine-tuning erases nearly all architectural differences introduced in the projection stage; (iii) the largest decoding lever, sentence completion, improves FENSE mostly by removing the fluency-error term rather than by improving semantics; and (iv) qualitatively the models capture genre, instrumentation and mood well but share a systematic bias toward hallucinating a vocalist. We frame the work as an efficiency- and analysis-oriented study of the minimum viable architecture for music captioning, not a state-of-the-art claim.

1. INTRODUCTION

Most audio captioning research targets *general* sound — environmental events and acoustic scenes — on AudioCaps [6] and Clotho [7]. Music is different: a useful music caption describes *musical* attributes such as genre, instrumentation, mood, tempo and structure rather than discrete sound events. This paper asks a deliberately minimal question: *can a lightweight language model understand music semantics through a simple projection from a frozen audio embedding?*

Our approach, CLAPCAP, adapts ClipCap [1], which captions images by projecting a frozen CLIP embedding into a prefix of pseudo-tokens for GPT-2. We replace the image encoder with a frozen CLAP audio encoder [4], caption music instead of images, and — crucially — turn the single-LM recipe into a *controlled comparison* of language models. Because the encoder, projection, data split and evaluation are all shared, the language model is the only thing that changes between conditions.

The contributions of this paper are:

- An audio adaptation of the ClipCap prefix-captioning recipe to music, with a single shared evaluation harness.
- A *noise-floor* methodology: per-model run-to-run variance from three seeds, so single-run ablation deltas are interpretable rather than anecdotal.
- A controlled comparison of a decoder-only (GPT-2) and an encoder–decoder (T5) LM, with ablations over projection architecture and decoding strategy.
- An honest qualitative analysis, including a reproducible vocalist-hallucination failure mode that we trace to the general-purpose audio frontend.

We state up front what this paper is *not*: it is not a state-of-the-art system, and we do not compare our numbers against systems evaluated on different datasets (Section 2). The value is the controlled analysis and the efficiency of the recipe — only a small projection (plus the LM) is trained.

2. RELATED WORK

Prefix captioning. ClipCap [1] maps a frozen CLIP image embedding to a constant-length prefix (K=10 tokens) consumed by GPT-2, training only a small mapping network. It offers two design points: a lightweight transformer mapper with a frozen LM, and an MLP mapper with a fine-tuned LM. CLAPCAP is the audio analogue of the MLP-mapper, fine-tuned-LM variant. Notably, ClipCap reports that this variant *overfits* as trainable capacity grows — an observation we recover, amplified, when scaling the LM (Section 6).

General audio captioning. Audio captioning is typically benchmarked on AudioCaps [6] and Clotho [7], which describe general sounds and provide multiple references per clip. These differ from music captioning in

domain, number of references, and the metrics commonly reported (CIDEr/SPIDEr rather than the semantic FENSE). Cross-dataset numerical comparison is therefore not meaningful, and we avoid it.

Music captioning. MusicCaps [2] is a hand-curated set of 5,521 ten-second clips from AudioSet, each annotated by musicians with a multi-sentence caption and a list of musical aspects. LP-MusicCaps [3] addresses caption scarcity by generating pseudo-captions with an LLM over tag datasets and trains a convolutional-encoder/BART-decoder captioner. Such domain-specific captioners are a natural *reference* for CLAPCAP, but a fair comparison must run them on our exact test split and avoid the leakage that arises when a captioner is itself trained on MusicCaps; we leave this to future work.

Audio language models. End-to-end audio LLMs such as Qwen-Audio [10] and SALMONN [11] couple a large LM to an audio encoder and are trained on large instruction corpora. They are far heavier than the present recipe; we regard them as an upper-bound reference direction rather than a controlled comparison, since they bypass the frozen-encoder/learned-prefix design we study.

3. METHOD

Overview. A clip is encoded once by a frozen CLAP model into a d_a-dimensional embedding. A projection network f_θ maps this vector to a prefix $p_1, \ldots, p_k \in \mathbb{R}^{d_{\text{lm}}}$ of k pseudo-tokens in the LM embedding space. The LM then generates the caption autoregressively, conditioned on the prefix (Figure 1).

Audio encoder. We use the LAION CLAP checkpoint `laion/clap-htsat-unfused` [4], a 512-dimensional audio embedding. CLAP is trained on general audio and is *not* music-specialised — a deliberate test of how far a general frontend carries on a music task, and a limitation we revisit. Embeddings are precomputed once and cached, so the encoder never runs during training or evaluation. The "unfused" variant ingests a fixed $\sim$10 s window, matching MusicCaps clip length.

Projection network. The projection is a small MLP, Linear $\to$ LayerNorm $\to$ GELU $\to$ Dropout $\to$ Linear, reshaped to (k, d_{lm}). The baseline uses prefix length k=8, dropout 0.3 and depth 2.

Language models. We compare GPT-2 [8] (decoder-only, 124M) and T5-base [9] (encoder–decoder, 220M). For T5 the prefix is fed to the encoder; for GPT-2 it is prepended as input embeddings. Extending the same harness to modern billion-parameter decoders is in progress (Section 6).

Two-stage training. In Stage 1 (S1) the LM is frozen and only the projection is trained; in Stage 2 (S2) the projection and the full LM are fine-tuned together (the audio encoder stays frozen). S1 corresponds to ClipCap's frozen-LM regime and S2 to its fine-tuned-LM regime. The objective in both stages is autoregressive cross-entropy on caption tokens conditioned on the prefix,

$$\mathcal{L} = -\sum_j \log p_\theta(c_j \mid p_1, \ldots, p_k, c_1, \ldots, c_{j-1}) .$$

We use AdamW (weight decay 0.1), cosine annealing, batch size 8 and early stopping (patience 5). Per model, only the learning rates are tuned, via a 16-trial Optuna search over an 8-epoch proxy; all else is fixed so the LM stays the controlled variable.

Efficiency. The only newly added module is the projection ($\sim$10.2M parameters): Stage 1 trains it alone with the LM frozen, and Stage 2 additionally fine-tunes the LM. The complete model is 124M (GPT-2) or 220M (T5) parameters — one to two orders of magnitude smaller than the billion-parameter end-to-end audio LLMs — and a full two-stage run trains in **[TODO: $\sim$X h]** on a single NVIDIA RTX 5090. Because CLAP embeddings are precomputed once, the audio encoder is never run during training.

4. EXPERIMENTAL SETUP

Data. We use MusicCaps [2] with a fixed three-way random split: a held-out test set (n=529, final metrics only), a validation set ($n\approx$495, for checkpoint selection and hyperparameter search), and the remainder for training ($n\approx$4,455). The split seed is fixed across all conditions, so every model and seed is scored on the identical test set.

Metrics. A single shared harness scores all conditions with the `aac-metrics` suite: BLEU-1, BLEU-4, METEOR, ROUGE-L, CIDEr-D, SPICE, SPIDEr, SBERT-sim, FER and FENSE. FENSE [5] is our primary metric: it combines a sentence-BERT semantic similarity with a fluency-error penalty (FER), making it sensitive to the semantic correctness a listener cares about. We apply no post-processing by default; sentence trimming is treated as an explicit decoding ablation (Section 5.4).

Noise floor. To judge whether a delta is real, we retrain each baseline with three seeds (42, 43, 44), varying only initialisation, shuffling and dropout while holding the data split fixed. The standard deviation of each metric across seeds is our *noise floor*; deltas smaller than it are not interpreted as effects.

5. RESULTS

5.1 Baseline comparison

Table 1 reports greedy-decoding test metrics with per-model noise floors. GPT-2 leads on every metric, including the primary FENSE (0.3998 vs. 0.3954), but the margin (0.004) is smaller than either model's FENSE noise floor ($\pm$0.0137 for GPT-2, $\pm$0.0345 for T5). We therefore do *not* read it as evidence that one architecture is better at music captioning under this recipe. The noise floor is also $\sim$2.5$\times$ larger for T5 than GPT-2, so the encoder–decoder is noticeably noisier run-to-run and each model must be judged against its *own* variance.

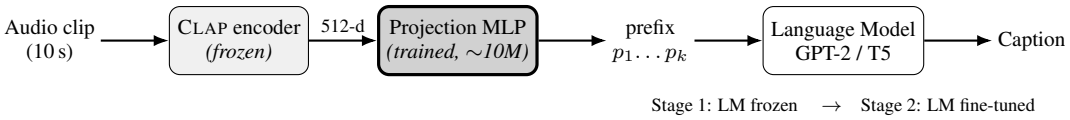

Figure 1. The CLAPCAP pipeline. A frozen CLAP encoder turns the audio clip into a single 512-d embedding; a small projection MLP (the only newly added module) maps it to a k-token prefix in the LM embedding space; the LM generates the caption. Stage 1 trains the projection with the LM frozen; Stage 2 additionally fine-tunes the LM.

Table 1. Baseline (greedy) test-set metrics for both language models, with per-model run-to-run standard deviation (noise floor) from three seeds. FENSE is the primary metric; best per column in bold.

Model	BLEU-1	BLEU-4	METEOR	ROUGE-L	CIDEr-D	SPICE	SPIDEr	SBERT-sim	FER	FENSE
GPT-2	**0.3091**	**0.0631**	**0.1265**	**0.2329**	**0.1109**	**0.1091**	**0.1100**	**0.5820**	**0.3478**	**0.3998 ±0.0137**
T5	0.3003	0.0574	0.1162	0.2275	0.0847	0.0866	0.0857	0.5564	0.3251	0.3954 ±0.0345

The GPT-2/T5 FENSE gap (0.004) is smaller than either model's noise floor, i.e. not significant.

5.2 Stage 1 vs. Stage 2

Figure 2 shows validation loss per epoch for every projection configuration. In Stage 1, where only the projection is trained, the configurations spread widely (a range of 0.38 for GPT-2 and 0.83 for T5). After Stage 2 fine-tuning that spread collapses to 0.03 for both models: full fine-tuning largely erases the architectural differences introduced by the projection. The S2 curves also reveal early overfitting — validation loss bottoms within a few epochs and then rises — which early stopping handles.

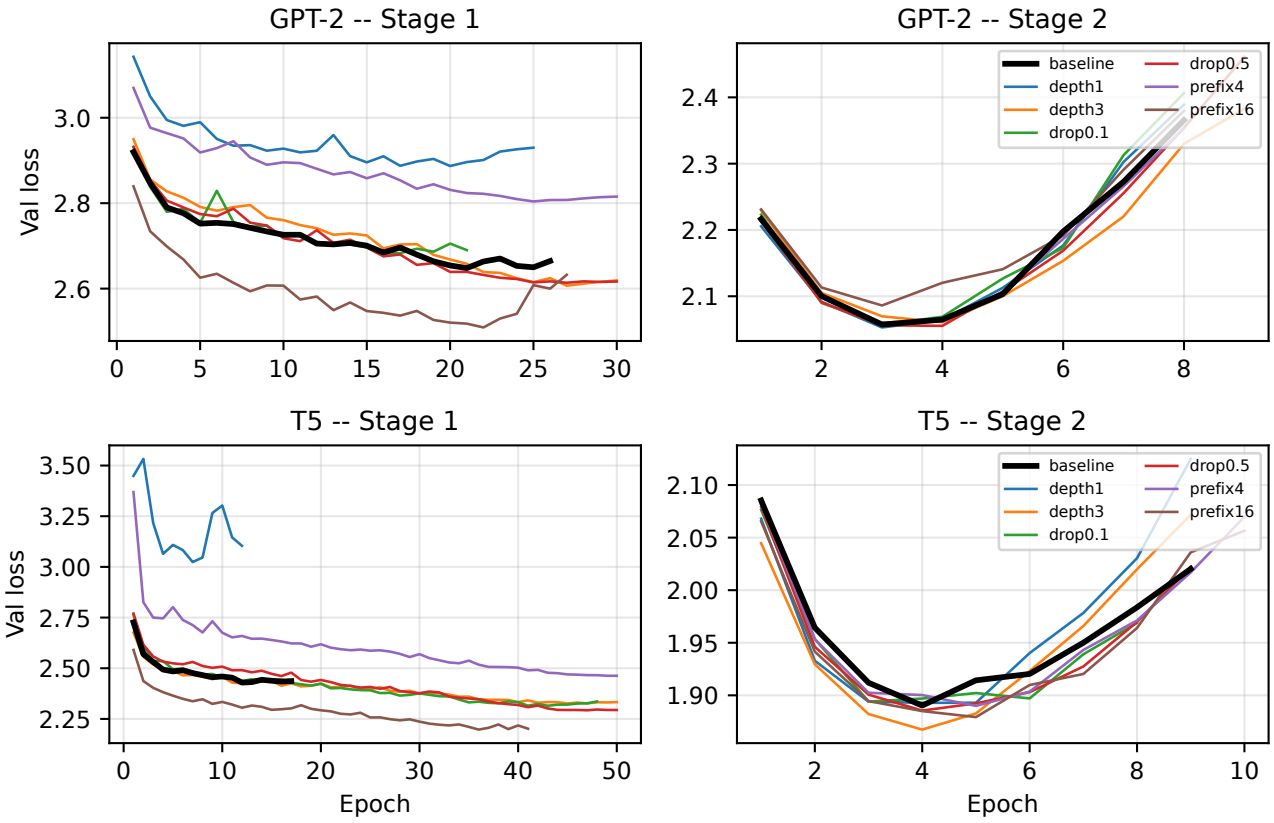

Figure 2. Validation loss per epoch for all projection configurations. *Left:* Stage 1 (projection only) shows a wide spread across configurations. *Right:* Stage 2 (full fine-tuning) collapses that spread — the configurations become nearly indistinguishable — and shows the early-overfitting onset. The baseline is drawn in bold.

5.3 Architecture ablations

We vary prefix length ($\{4, 16\}$ around 8), dropout ($\{0.1, 0.5\}$ around 0.3) and projection depth ($\{1, 3\}$ around 2), retraining each from scratch with greedy decoding. Figure 3 plots the resulting FENSE against the baseline $\pm$ noise-floor band. Consistent with Section 5.2, most configurations fall within the band: no projection hyperparameter reliably moves the primary metric for GPT-2. For T5 the prefix variants produce larger FENSE swings, but — as the next section shows — these coincide with collapses in the fluency-error term rather than genuine semantic gains.

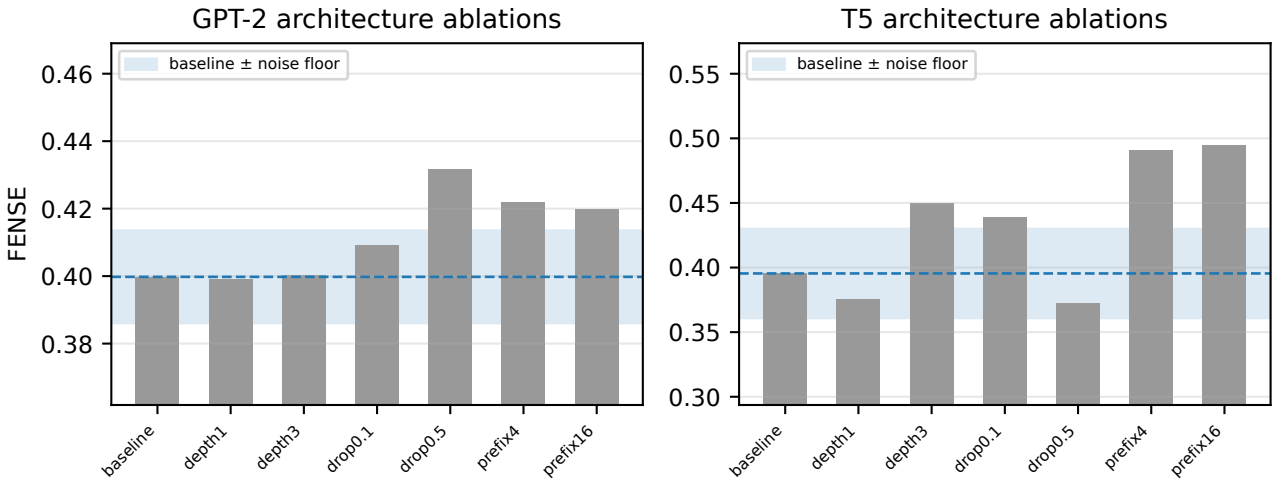

Figure 3. Architecture-ablation FENSE per configuration. The shaded band is the baseline ± one noise-floor standard deviation. Bars inside the band are not distinguishable from the baseline.

5.4 Decoding ablations

Decoding strategies are evaluated on the fixed baseline checkpoint (no retraining), isolating generation behaviour. Table 2 lists the top configurations by FENSE. The dominant lever is trimming trailing incomplete sentences: it lifts FENSE from 0.3998 to 0.5796 for GPT-2 and from 0.3954 to 0.5762 for T5.

This gain is, however, *largely an artifact of the metric*. Figure 4 plots FENSE against FER across all decoding configurations: the relationship is almost perfectly inverse, and the trimmed configurations simply drive FER to near zero while the lexical metrics (CIDEr-D, SPIDEr in Table 2) barely move. In other words, trimming removes the fluency penalty inside FENSE without making captions semantically more correct. We report it as a useful decoding choice for producing clean captions, but caution against reading the headline FENSE jump as a semantic improvement.

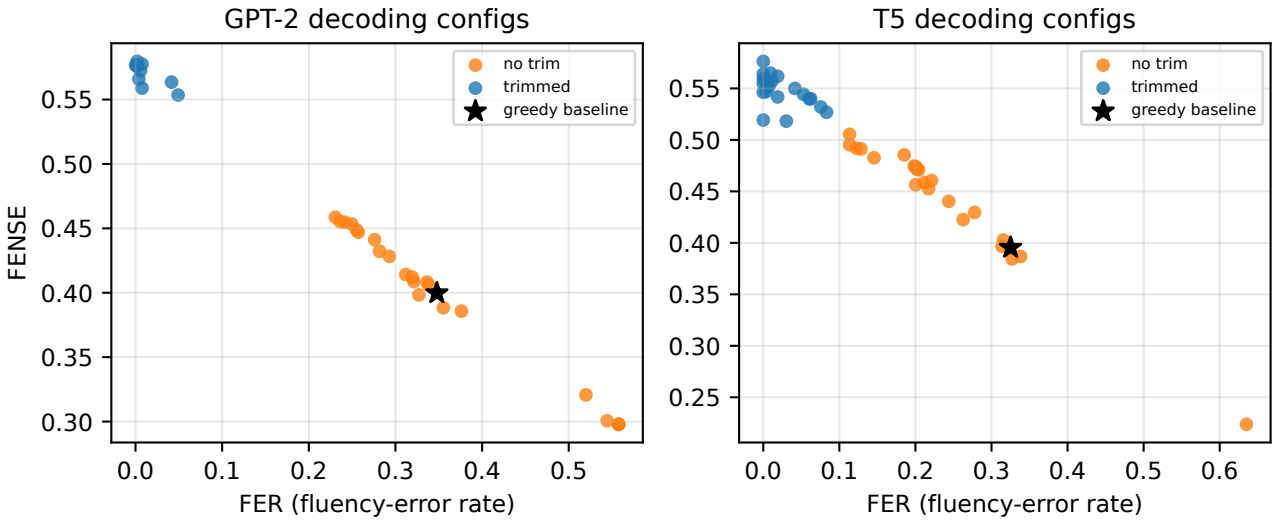

Figure 4. FENSE vs. FER over all decoding configurations. The near-perfect inverse trend shows that FENSE here is dominated by the fluency-error term: trimmed configurations (low FER) score high FENSE without improving semantic similarity. The star is greedy decoding.

Table 2. Top decoding configurations per model (eval-only on the baseline checkpoint), ranked by FENSE. Note that high FENSE coincides with near-zero FER while lexical metrics (CIDEr-D, SPIDEr) barely move – see Figure 4.

Config	FENSE	SBERT-sim	FER	CIDEr-D	SPIDEr
GPT-2					
greedy (baseline)	0.3998	0.5820	0.3478	0.1109	0.1100
trim_ngram2	**0.5796**	0.5807	0.0019	0.1092	0.1087
trim_rep1.2_ngram2_t0.3	0.5775	0.5812	0.0076	0.1003	0.1033
trim_rep1.2_ngram2	0.5766	0.5766	0.0000	0.0959	0.1000
trim_rep1.2_ngram3	0.5766	0.5781	0.0019	0.0926	0.0986
trim_rep1.2	0.5758	0.5774	0.0019	0.0924	0.0983
trim_rep1.3	0.5723	0.5748	0.0057	0.0885	0.0938
T5					
greedy (baseline)	0.3954	0.5564	0.3251	0.0847	0.0857
trim_rep1.2	**0.5762**	0.5762	0.0000	0.0863	0.0892
trim_beam5_ngram2	0.5649	0.5698	0.0095	0.0878	0.0945
trim_rep1.3	0.5640	0.5640	0.0000	0.0779	0.0883
trim_rep1.2_beam5_ngram2	0.5618	0.5716	0.0189	0.0985	0.1004
trim_rep1.2_ngram2	0.5593	0.5593	0.0000	0.0662	0.0831
trim_rep1.2_ngram2_t0.3	0.5591	0.5599	0.0019	0.0611	0.0792

5.5 Qualitative analysis

Table 3 shows representative successes and failures. The models reliably recover high-level musical content — genre (rock, jazz, electronic), instrumentation (electric guitar, bass, drums, piano) and mood (groovy, energetic, emotional). Two failure modes recur: on *non-music* audio the pipeline still emits a confident music caption, and greedy decoding occasionally degenerates into verbatim repetition. Most strikingly, both models default to describing "a male vocalist singing melodically" even when the reference is instrumental or explicitly states there is no singer. We attribute this vocal-presence blind spot to the general-purpose audio frontend and the strong language prior of the caption template.

Table 3. Representative successes and failures (baseline greedy; captions trimmed at the last full sentence for display). The models recover genre, instrumentation and mood, but default to a male vocalist even on instrumental or non-music audio.

	Successes
Reference	instrumental rock music with a drum kit, lots of cymbal crashes, acoustic guitar strumming, harmonized lead guitars and an active bass guitar.
GPT-2	This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing the main tune while the bass guitar is playing in the background. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy.
T5	This is a rock music piece. There is a male vocalist singing melodically. The electric guitar is playing a simple tune with a bass guitar playing in the background. The rhythmic background consists of a fast-paced acoustic drum beat.
Reference	This is a drum & bass electronic music piece. There is no singer. There is a strong and fuzzy feeling bass sound. The rhythmic background consists of a loud and high tempo electronic drum beat. The atmosphere is modern and energetic. This piece could be playing at a nightclub or a dance club.
GPT-2	This is a techno music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass. The rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is groovy. This piece could be used in the background of nightclubs.
T5	This is a techno house music piece. There is a male vocalist singing melodically. The melody is being played by the synth bass while the rhythmic background consists of a fast-paced electronic drum beat. The atmosphere is energetic.
Reference	A female singer sings this emotional melody. The song is medium tempo with guitar strumming rhythm, strong bass line, steady drumming rhythm, keyboard accompaniment and sound effects. The song is emotional and romantic. The song is a euro pop love song and has poor audio quality.
GPT-2	A female vocalist sings this soft melody. The tempo is medium with a keyboard accompaniment, steady drumming, steady bass line and keyboard accompaniment. The song is mellow, soft, mellifluous, emotional, sentimental, melancholic and romantic.
T5	This is a mellow, soft rock piece. There is a male vocalist singing melodically. The melody is being played by the electric guitar while the bass guitar is playing in the background. The rhythm is provided by a simple acoustic drum beat.
Reference	The low quality recording features a live performance of a jazz song and it consists of a groovy bass guitar, groovy rhodes keys, electric guitar melody, shimmering hi hats, punchy kick and punchy snare hits. It sounds groovy, funky and passionate. The recording is noisy and in mono.
GPT-2	This is a jazz music piece. There is a saxophone solo being played. The melody is being played by the piano and the electric guitar. The rhythmic background consists of a simple acoustic drum beat. The atmosphere is groovy. This piece could be used in the background of a jazz bar.
T5	This is a live recording of a jazz music piece. There is a saxophone playing the lead melody while a bass guitar is playing in the background. The rhythm is being played by the acoustic drums. The atmosphere is jazzy.
	Failure cases
Reference	The low quality recording features a printer that produces printing sounds and some muffled music playing in the background, probably on a radio. It is noisy and in mono.
GPT-2	This is a recording of a live performance of a pop music piece. There is a male vocalist singing melodically. The melody is being played by a keyboard and a bass guitar while the rhythmic background is provided by a fast tempo electronic drum beat.
T5	This is a live recording of a DJ performance. The DJ is scratching the turntable repeatedly. The turntable is a low quality recording. The sound of the turntable is distorted and the recording is of very poor quality.
Reference	The low quality recording features a metallic hit, followed by resonating bowl tone. The recording is mono and noisy.
GPT-2	This is a meditation music piece. There is a bowl-like sound being played by the bowl. The bowl is being rubbed repeatedly. There is a calming and relaxing atmosphere to this piece. This piece can be used in the background of meditation videos. It can also be used in the background of meditation videos.
T5	This is a Tibetan singing bowl playing in the background of a meditation video. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played. There is a chanting bowl being played.

6. LIMITATIONS AND FUTURE WORK

General-purpose audio frontend. Our encoder is trained on general audio, not music. A music-specialised encoder (a music CLAP checkpoint, MERT or MusicFM) is a natural and likely beneficial swap, especially for the fine-grained attributes (vocal presence, instrument identity) where the model is weakest.

Capacity vs. data. Scaling the LM is not free in this low-data regime ($\sim$4.5k clips). In preliminary runs a billion-parameter decoder fine-tuned end-to-end overfits within a single epoch — the same MLP-plus-fine-tuning overfitting that ClipCap [1] reports, amplified by scale — and requires a gentler schedule to remain comparable. This supports the lightweight framing and motivates our in-progress extension to such models.

Metric caveats. MusicCaps provides a single reference per clip, which mechanically depresses n-gram metrics relative to multi-reference benchmarks. And as Section 5.4 shows, FENSE can be inflated by fluency-driven trimming; we mitigate this by reporting FER and lexical metrics alongside it.

Reference and human evaluation. We do not yet compare against a domain-specific music captioner on our split; the clean route is a leakage-free checkpoint evaluated on our exact test items via the same harness. We also plan an LLM-as-judge and human evaluation of semantic correctness, reporting inter-rater agreement.

7. CONCLUSION

We presented CLAPCAP, a controlled study of lightweight language models for *music* captioning via an audio adaptation of the ClipCap prefix recipe. Holding the frozen CLAP encoder, the projection, the data split and the evaluation fixed, we found that a decoder-only and an encoder–decoder LM are statistically indistinguishable at baseline once run-to-run variance is measured; that second-stage fine-tuning erases projection-level differences; and that the largest decoding lever improves the primary metric mostly

by removing a fluency penalty rather than by improving semantics. A reproducible vocalist hallucination points to the general-purpose audio frontend as the next thing to improve. The results characterise the minimum viable architecture for music captioning and the care needed to interpret its metrics, rather than chasing state of the art.

8. REFERENCES

[1] R. Mokady, A. Hertz, and A. H. Bermano, "ClipCap: CLIP Prefix for Image Captioning," *arXiv:2111.09734*, 2021.

[2] A. Agostinelli, T. I. Denk, Z. Borsos, *et al.*, "MusicLM: Generating Music From Text," *arXiv:2301.11325*, 2023.

[3] S. Doh, K. Choi, J. Lee, and J. Nam, "LP-MusicCaps: LLM-Based Pseudo Music Captioning," in *Proc. ISMIR*, 2023.

[4] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation," in *Proc. ICASSP*, 2023.

[5] Z. Zhou, Z. Zhang, X. Xu, *et al.*, "Can Audio Captions Be Evaluated with Image Caption Metrics?," in *Proc. ICASSP*, 2022.

[6] C. D. Kim, B. Kim, H. Lee, and G. Kim, "AudioCaps: Generating Captions for Audios in The Wild," in *Proc. NAACL-HLT*, 2019.

[7] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An Audio Captioning Dataset," in *Proc. ICASSP*, 2020.

[8] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, Tech. Rep., 2019.

[9] C. Raffel, N. Shazeer, A. Roberts, *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *JMLR*, vol. 21, no. 140, 2020.

[10] Y. Chu, J. Xu, X. Zhou, *et al.*, "Qwen-Audio: Advancing Universal Audio Understanding via Unified Large-Scale Audio-Language Models," *arXiv:2311.07919*, 2023.

[11] C. Tang, W. Yu, G. Sun, *et al.*, "SALMONN: Towards Generic Hearing Abilities for Large Language Models," in *Proc. ICLR*, 2024.